

UNIVERSITE DE
YAOUNDE I

ECOLE NATIONALE
SUPERIEURE
POLYTECHNIQUE
DE YAOUNDE

DEPARTEMENT DE
GENIE
INFORMATIQUE

HUMANITES
NUMERIQUES



THE UNIVERSITY
OF YAOUNDE I

NATIONAL
ADVANCED SCHOOL
OF ENGINEERING
OF YAOUNDE

DEPARTMENT OF
COMPUTER
ENGINEERING

DIGITAL
HUMANITIES

ANALYSE DE L'IMPACT DES THREADS VS PROCESSUS SUR LA CONSOMMATION DE CACHE L1/L2

Rapport de projet

Groupe 12

Sous l'encadrement de :

Mme CHIASSEUP

Membres du groupe :

ABOUBAKAR KARIM	24P893
ABOUBAKAR HUDOU	24P831
ABDELAZIZ DALIL	24P891 (Chef de groupe)
AKAMBA PHAREL	24P941
AMELI TSOWA	24P942
ATANGANA ALOGO	23P064
ATANGANA HENRY	24P897

Membres du Groupe 12

Nom & Prénom	Matricule	Rôle
ABOUBAKAR KARIM	24P893	Membre
ABOUBAKAR HUDOU	24P831	Membre
ABDELAZIZ DALIL	24P891	Chef de groupe
AKAMBA PHAREL	24P941	Membre
AMELI TSOWA	24P942	Membre
ATANGANA ALOGO	23P064	Membre
ATANGANA HENRY	24P897	Membre

Encadrante : **Mme CHIASSEUP**

Département de Génie Informatique — ENSPY

Année académique 2024–2025

Contents

1	Introduction	3
2	Généralités sur les notions de threads, processus et cache L1/L2	4
2.1	Le processus	4
2.1.1	Définition, caractéristique et objectifs	4
2.1.2	Code de création d'un processus	5
2.2	Les threads	7
2.2.1	Définition, caractéristique, objectifs et usage concret	7
2.2.2	Notion de traitement multithread	8
2.2.3	Code de création d'un thread	8
2.2.4	Différence/ressemblance entre thread et processus	10
2.3	La cache L1/L2	10
2.3.1	Définition, caractéristique, hiérarchisation (L1–L2–L3...) et ob- jectifs	10
2.3.2	Circuit d'utilisation de la cache L1/L2 par un processus	11
3	La méthodologie expérimentale	15
3.1	Environnement et outils d'utilisation	15
3.2	Étapes de déroulement	16
3.2.1	1 ^{ère} étape : préparer les codes pour les processus et thread	16
3.2.2	2 ^{ième} étape : Prélèvement des données et constitution d'un datasheet	18
3.2.3	3 ^{ième} étape : Analyse statistique exploratoire du datasheet à travers python	19
3.2.4	4 ^{ième} étape : Commentaire et discussion	19
3.3	Mise en œuvre	19
4	Résultats et discussion	23
4.1	Discussion	29
4.1.1	Le thread est l'unité reine de la performance pure	29
4.1.2	Le processus est le choix de la sécurité et de la robustesse	30
4.1.3	La localité des données dicte la vitesse moderne	30
5	Conclusion	31

1 Introduction

Les systèmes informatiques contemporains reposent sur l'exécution concurrente d'un grand nombre de tâches, qu'il s'agisse de processus indépendants ou de threads appartenant à une même application. Afin d'assurer le partage du processeur entre ces différentes entités d'exécution, le système d'exploitation effectue régulièrement des changements de contexte (*context switches*), consistant à interrompre temporairement l'exécution d'une tâche afin d'en exécuter une autre. Bien qu'indispensables au fonctionnement des systèmes multitâches, ces opérations introduisent un surcoût lié notamment à la sauvegarde et à la restauration des états d'exécution.

Parallèlement, les performances des processeurs modernes reposent largement sur l'efficacité de la hiérarchie mémoire, dont le rôle est de réduire la latence des accès aux données grâce à l'exploitation des principes de localité temporelle et spatiale. Toutefois, lors d'un changement de contexte, les données et instructions chargées dans les caches par une tâche peuvent être remplacées par celles de la tâche suivante, réduisant ainsi l'efficacité des caches et augmentant le nombre de défauts mémoire (*cache misses*). Cette dégradation est susceptible d'affecter significativement les performances globales des applications, en particulier dans les environnements fortement concurrents.

Dans le cadre du projet consacré à l'étude de l'impact des changements de contexte sur les performances de la hiérarchie mémoire des processeurs modernes, nous nous intéressons aux interactions entre les mécanismes d'ordonnancement des systèmes d'exploitation et le comportement des mémoires caches de premier et de second niveau (L1 et L2).

Dans ce contexte, la problématique de ce projet peut être formulée de la manière suivante :

Dans quelle mesure les changements de contexte influencent-ils les taux de défaut des caches L1 et L2, et comment cet impact varie-t-il selon que l'exécution repose sur 2 processus ou sur 2 threads ?

Afin d'apporter des éléments de réponse à cette question, une étude expérimentale a été menée en faisant varier de manière contrôlée la fréquence des changements de contexte au moyen de la primitive d'ordonnancement `sched_yield()`. Les performances ont été mesurées à l'aide des compteurs matériels fournis par l'outil `perf`, permettant de quantifier simultanément le nombre de changements de contexte ainsi que les défauts observés aux différents niveaux de la hiérarchie mémoire.

2 Généralités sur les notions de threads, processus et cache L1/L2

L'évolution des architectures informatiques modernes repose sur une gestion toujours plus fine des ressources matérielles et logicielles. Au cœur de cette organisation se trouvent trois notions fondamentales : les processus, les threads et les mémoires caches (L1/L2). Ces concepts, intimement liés, constituent les piliers de tout système d'exploitation et de toute architecture processeur contemporaine.

Cette partie se propose d'explorer en détail chacune de ces notions, leurs caractéristiques, leurs interactions et leur rôle dans l'optimisation des performances des systèmes modernes.

2.1 Le processus

2.1.1 Définition, caractéristique et objectifs

Un processus représente l'unité de base de l'exécution d'un programme en cours. C'est une instance d'un programme chargée en mémoire centrale, disposant de son propre espace d'adressage, de ses ressources et de son contexte d'exécution.

Un processus est caractérisé par de nombreux points (éléments d'entrée, valeur ajoutée, élément de sortie...) mais les principaux restent son état et ses ressources.

S'agissant de son état, les règles d'applications sont les suivantes :

- Quand un processus s'exécute, il change d'état.
- Chaque processus peut se trouver dans chacun des états suivants :
 - **En exécution** : Les instructions sont en cours d'exécution (en train d'utiliser la CPU).
 - **En attente** : Le processus attend qu'un événement se produise.
 - **Prêt** : Le processus attend d'être affecté à un processeur.
- Un seul processus peut être en exécution sur n'importe quel processeur à tout moment. Toutefois, plusieurs processus peuvent être prêts et en attente.

Sous les systèmes Unix/Linux, chaque processus est identifié par un numéro unique appelé PID (*Process Identifier*). Le système d'exploitation maintient une table des processus permettant de gérer l'ensemble des processus actifs. La commande `ps` ou `top` sur linux permet d'afficher l'ensemble des processus en cours d'exécution.

```
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ top
top - 02:05:28 up 16 min, 1 user, load average: 0,91, 0,92, 0,67
Tâches: 266 total, 2 en cours, 264 en veille, 0 arrêté, 0 zombie
%Cpu(s): 0,9 ut, 1,2 sy, 0,0 ni, 97,2 id, 0,4 wa, 0,0 hi, 0,3 si, 0,0 st
MiB Mem : 7836,3 total, 4086,5 libr, 2184,3 util, 1889,3 tamp/cache
MiB Éch: 4096,0 total, 4096,0 libr, 0,0 util, 5652,0 dispo Mem
```

PID	UTIL.	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TEMPS+	COM.
3205	dalil-l+	20	0	3230188	412792	206816	S	4,3	5,1	2:09.90	firefox
348	root	20	0	0	0	0	I	1,0	0,0	0:50.92	kworker/u8:6-events_unbound
353	root	20	0	0	0	0	I	1,0	0,0	0:50.58	kworker/u8:8-events_unbound
4777	dalil-l+	20	0	2474908	142920	90752	S	1,0	1,8	0:00.75	Isolated Web Co
2224	dalil-l+	20	0	4336024	285820	137252	S	0,7	3,6	1:03.56	gnome-shell
24	root	20	0	0	0	0	S	0,3	0,0	0:00.34	ksoftirqd/1
188	root	20	0	0	0	0	I	0,3	0,0	0:04.72	kworker/2:2-mm_percpu_wq
3149	dalil-l+	20	0	638536	60920	47216	S	0,3	0,8	0:03.04	gnome-terminal-

Les objectifs d'un processus sont centraux pour un système d'exploitation. En effet, il a pour objectif de contrôler l'utilisation des ressources système et de masquer la complexité matérielle aux utilisateurs et développeurs. Il permet également la bonne marche de la machine grâce à des objectifs fondamentaux bien précis.

2.1.2 Code de création d'un processus

Normalement un processeur ne peut exécuter qu'un processus à la fois par cœur, mais fonctionnant de cette manière le temps d'exécution global reste très lent et on fait face à des problèmes d'optimisation (il serait par exemple impossible d'avoir plusieurs onglets ouverts dans notre navigateur dans ce cas). Pour régler ce problème, une technique dites changement de contexte a été mise sur pied. L'idée est de traiter plusieurs processus en même temps en faisant des interruption régulière (qui sont enregistré dans les registres processus). Un temps d'exécution définis (par l'ordonnanceur) pour le processus A, un autre temps pour le processus B et ainsi de suite. C'est comme lire plusieurs livres en même temps, en marquant continuellement le numéro des pages de chaque livre pour se retrouver lors de la prochaine lecture.

Dans notre cas d'utilisation, une commande utile sous Linux, permet de créer un processus enfant à partir d'un processus parent. La commande `fork()`. Lorsque le traitement se fait dans le processus père, `fork()` renvoie un nombre supérieur à 1. Lorsque le traitement se fait dans le processus fils, `fork()` renvoie 0.

```

C:\Users\BAMDAM\Desktop\polytech doc\HN-2\SE\code creation processus.c
2  #include <unistd.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5
6  int main()
7  {
8      pid_t pid = fork();
9
10     if (pid < 0)
11     {
12         perror("fork");
13         return 1;
14     }
15
16     if (pid == 0)
17     {
18         // Processus fils
19         printf("Je suis le processus fils\n");
20         printf("PID = %d\n", getpid());
21
22         for (long i = 0; i < 1000000000L; i++);
23
24         printf("Fin du processus fils\n");
25     }
26     else
27     {
28         // Processus père
29         printf("Je suis le processus père\n");
30         printf("PID = %d\n", getpid());
31         printf("PID du fils = %d\n", pid);
32
33         for (long i = 0; i < 1000000000L; i++);
34
35         printf("Fin du processus père\n");
36
37         wait(NULL);
38     }

```

Code C de création du processus

Dans ce code, on duplique le processus lié à l'exécution du code via la commande `fork()` (ligne 8) et on fait une instruction de condition. Si le processeur traite le processus père, il sera exécuté les lignes 19–24 (ou une partie de ces lignes via l'ordonnanceur). Dans le cas contraire, il sera exécuté les lignes 29–35 (ou une partie en alternance de ces lignes).

NB : la boucle `for()` est utilisé pour créer de la charge de traitement au processeur pour avoir un petit laps de temps d'observation.

Pour exécuter le programme sur linux, il est nécessaire de télécharger le compilateur du langage C GCC (`sudo apt install gcc`). Rendre le code exécutable avec la commande :

```
-- gcc creation_processus.c -o creation_processus
```

Et enfin entrer `./creation_processus`


```
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ gcc creation_processus.c -o process
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ ./process
Je suis le processus père
PID = 4327
PID du fils = 4328
je suis le processus filsPID = 4328
Fin de processus père
Fin du processus fils
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ ./process
```

2.2 Les threads

2.2.1 Définition, caractéristique, objectifs et usage concret

Un thread (ou fil d'exécution) est la plus petite unité d'instructions qu'un système d'exploitation peut planifier et exécuter. Souvent qualifié de « processus léger », il permet à une application d'effectuer plusieurs tâches en parallèle au sein du même espace mémoire.

- **Threads utilisateur** : Gérés par des bibliothèques au niveau de l'espace utilisateur (sans l'intervention directe du noyau). Ils sont très rapides, mais si un thread bloque, tout le processus s'arrête.
- **Threads noyau** : Directement pris en charge et gérés par le système d'exploitation. Ils sont plus robustes (le blocage d'un thread n'affecte pas les autres) mais nécessitent plus de temps pour les créations et commutations.

Pour voir les threads en exécution sur une machine linux, on peut utiliser la commande `top -H`.

Ils sont généralement utiles pour 2 raisons :

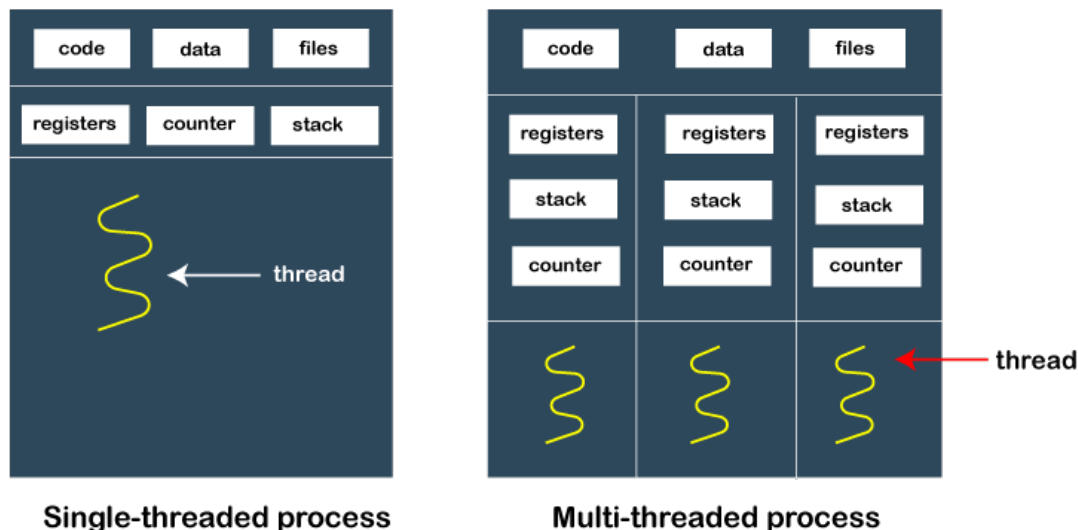
- **Performances** : Le partage de données en mémoire permet des communications rapides, bien plus efficaces que les systèmes de communication inter-processus (IPC).
- **Réactivité** : Un thread peut continuer à traiter les interactions utilisateur pendant qu'un autre effectue un calcul lourd en arrière-plan.

Dans les systèmes d'exploitation modernes, les threads sont indispensables dans de nombreux scénarios :

- **Traitement de texte** : Un thread gère l'interface utilisateur (saisie), un autre vérifie l'orthographe en tâche de fond, et un troisième sauvegarde le document périodiquement.
- **Serveurs Web** : Un thread principal écoute les nouvelles connexions. Dès qu'un client arrive, un thread dédié est créé pour traiter sa requête (pages, images) sans bloquer les autres utilisateurs.

- **Jeux vidéo** : Ils permettent de séparer les charges. Un thread calcule la physique, un autre gère l'intelligence artificielle (IA), et une troisième traite l'affichage graphique.

2.2.2 Notion de traitement multithread



Le traitement multithread (ou multithreading) est une technique informatique qui permet à un processus unique de diviser son exécution en plusieurs threads. Permettant au processeur de gérer ces flux d'instructions simultanément, améliorant ainsi considérablement les performances et la réactivité du système. Comme expliqué dans le cas des processus plus haut, Le multithreading est comparable à la capacité de lire plusieurs chapitres en même temps, en alternant rapidement de l'un à l'autre, vous donnant l'impression que la lecture globale avance simultanément.

Cela a des avantages de performances sans précédents sur le CPU mais représente des défis très difficile à gérer :

- **Accès concurrents** : Si deux threads essaient de modifier la même zone mémoire en même temps, cela peut entraîner des erreurs ou des pertes de données.
- **Synchronisation** : Les développeurs doivent utiliser des mécanismes de verrouillage (comme les mutexes) pour s'assurer que les threads s'exécutent de manière ordonnée et sécurisée.

2.2.3 Code de création d'un thread

Pour créer un thread en langage C, il est primordial de suivre une norme dites norme POSIX. Voici les étapes :

- **Inclure la bibliothèque** : Vous devez ajouter `#include <pthread.h>` en haut de votre fichier.
- **Compiler avec l'option** : Lors de la compilation, ajoutez l'option `-pthread` (ex: `gcc programme.c -o programme -pthread`).

- **Fonction `pthread_create`** : La fonction prend 4 paramètres :
 - Un pointeur vers l'identifiant du thread (`pthread_t`).
 - Les attributs du thread (NULL pour les paramètres par défaut).
 - Le nom de la fonction que le thread doit exécuter.
 - L'argument unique à passer à cette fonction (NULL si aucun).
- **Attendre la fin du thread** : Utilisez `pthread_join()` pour forcer le processus principal à attendre la fin de l'exécution du thread avant de quitter.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  // La fonction qui sera exécutée par le thread
6  void *fonction_du_thread(void *arg) {
7      printf("Le thread est en cours d'exécution !\n");
8      return NULL;
9  }
10
11 int main() {
12     pthread_t mon_thread;
13
14     // Création du thread
15     if (pthread_create(&mon_thread, NULL, fonction_du_thread, NULL) != 0) {
16         perror("Erreur lors de la création du thread");
17         return 1;
18     }
19
20     // Attente de la fin du thread
21     pthread_join(mon_thread, NULL);
22     printf("Le thread est terminé.\n");
23
24     return 0;
25 }
```

Script C de création d'un petit Thread

NB : La syntaxe de déclaration de fonction à la ligne 6 fait également partis de l'imposition de la norme. L'idée est que comme le système ne peut pas deviner à l'avance ce que vous voulez envoyer au thread (un simple numéro, une structure complexe, un tableau...) il utilise `void*` pour rester universel en pointant uniquement sur des adresses mémoire.

```
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ gcc creation_thread.c
-o thread -pthread
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ ./thread
Le thread est en cours d'exécution !
Fin du traitement du thread.
```

2.2.4 Différence/ressemblance entre thread et processus

La ressemblance principale est la suivante : un processus et un thread sont tous deux des unités d'exécution gérées par le système d'exploitation pour accomplir des tâches.

Les différences sont consignées dans le tableau suivant :

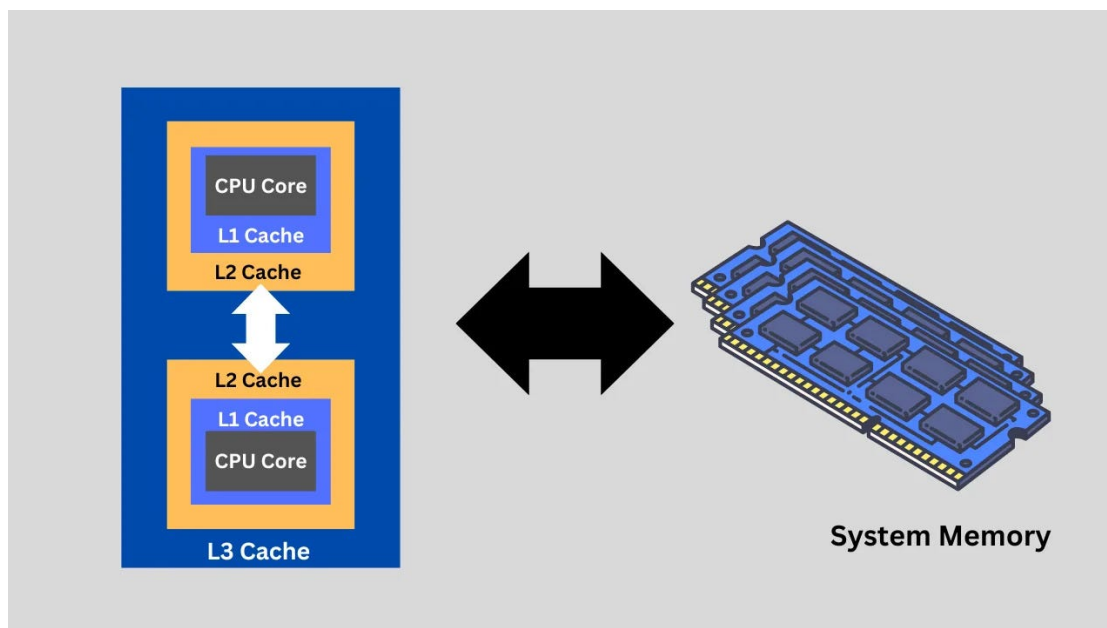
Critère	Processus	Thread
Définition	Programme en cours d'exécution	Sous-tâche légère d'un processus
Mémoire	Espace mémoire isolé	Partage la mémoire du processus parent
Création	Lourde et lente	Légère et rapide
Communication	Difficile (IPC)	Très facile (mémoire partagée)
Impact crash	Les autres processus continuent	Tout le processus parent plante

2.3 La cache L1/L2

2.3.1 Définition, caractéristique, hiérarchisation (L1–L2–L3...) et objectifs

La mémoire cache est une mémoire ultra-rapide placée entre le processeur (CPU) et la mémoire principale (RAM). Son objectif principal est de stocker les données et instructions fréquemment utilisées afin de réduire le temps d'attente du processeur et d'accélérer l'exécution des programmes. Voici un tableau récapitulatif :

	Position	Vitesse	Capacité	Rôle
L1	Dans chaque cœur	1–3 cycles	16–128 Ko	Données/instructions critiques
L2	Dédié par cœur	3–10 cycles	256 Ko – qq Mo	Tampon pour L1
L3	Partagé entre cœurs	Plus lent	4–96 Mo	Alimente L2, échanges inter-cœurs



Le fonctionnement de ces mémoires suit une hiérarchie stricte : lorsqu'une donnée est requise, le processeur consulte d'abord le L1. S'il ne l'y trouve pas (on parle de *cache miss*), il interroge le L2, puis le L3, et enfin la RAM.

2.3.2 Circuit d'utilisation de la cache L1/L2 par un processus

En lecture (*cache Miss*, *cache Hit*) — En écriture (*Write-through*, *Write-back*).

Le circuit d'utilisation des caches L1 et L2 dépend de l'opération processeur. En lecture (recherche de données), le système réagit selon qu'il y a un Hit (donnée présente) ou un Miss (donnée absente). En écriture (modification de données), il applique des stratégies de cohérence (*Write-through* ou *Write-back*).

La ligne de cache est lue d'abord depuis la RAM, passe par le cache L3, puis par le cache L2, pour enfin être copiée dans le cache L1.

Opération en lecture Lorsqu'un processus demande une donnée en mémoire, 2 cas se présentent :

- **Cache Hit (Succès) :**
 - La donnée est trouvée dans le cache L1. Le processeur l'utilise directement.
 - Si la donnée n'est pas dans le L1 mais dans le L2, elle est généralement copiée du L2 vers le L1 avant d'être transmise au processeur.
- **Cache Miss (Défaut) :**
 - La donnée est absente des caches. Le processeur doit aller la chercher dans la mémoire principale (RAM) ou dans le cache L3.
 - La donnée récupérée est ensuite stockée dans le cache L1 (et souvent dans le L2) pour être accessible plus rapidement lors des prochaines utilisations.

Opération en écriture Lorsqu'un processus modifie une donnée, le processeur doit s'assurer que la copie dans le cache et la copie dans la mémoire principale restent cohérentes. Il existe deux circuits principaux :

- **Write-through (Écriture simultanée) :**
 - Le processeur écrit la donnée modifiée à la fois dans le cache (L1/L2) et dans la mémoire principale (RAM).
 - Avantage : Les données sont toujours à jour en mémoire.
 - Inconvénient : Plus lent, car chaque écriture nécessite un accès à la RAM.
- **Write-back (Écriture différée) :**
 - Le processeur écrit la donnée modifiée uniquement dans le cache. L'emplacement mémoire modifié est alors marqué comme « sale » (*dirty*).
 - La mise à jour de la mémoire principale n'est effectuée que plus tard, lorsque la ligne de cache doit être écrasée par une nouvelle donnée et doit être expulsée.
 - Avantage : Beaucoup plus rapide.
 - Inconvénient : Le système est plus complexe à gérer car la RAM n'est pas constamment à jour.

Si linux, la commande `lscpu` permet de voir dans sa sortie les paramètres liés aux caches de manière laconique.

```
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ lscpu | grep L
Boutisme : Little Endian
Liste de processeur(s) en ligne : 0-3
Cache L1d : 128 KiB (4 instances)
Cache L1i : 128 KiB (4 instances)
Cache L2 : 1 MiB (4 instances)
Cache L3 : 6 MiB (1 instance)
Vulnerability L1tf: Mitigation; PTE Inversion
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$
```

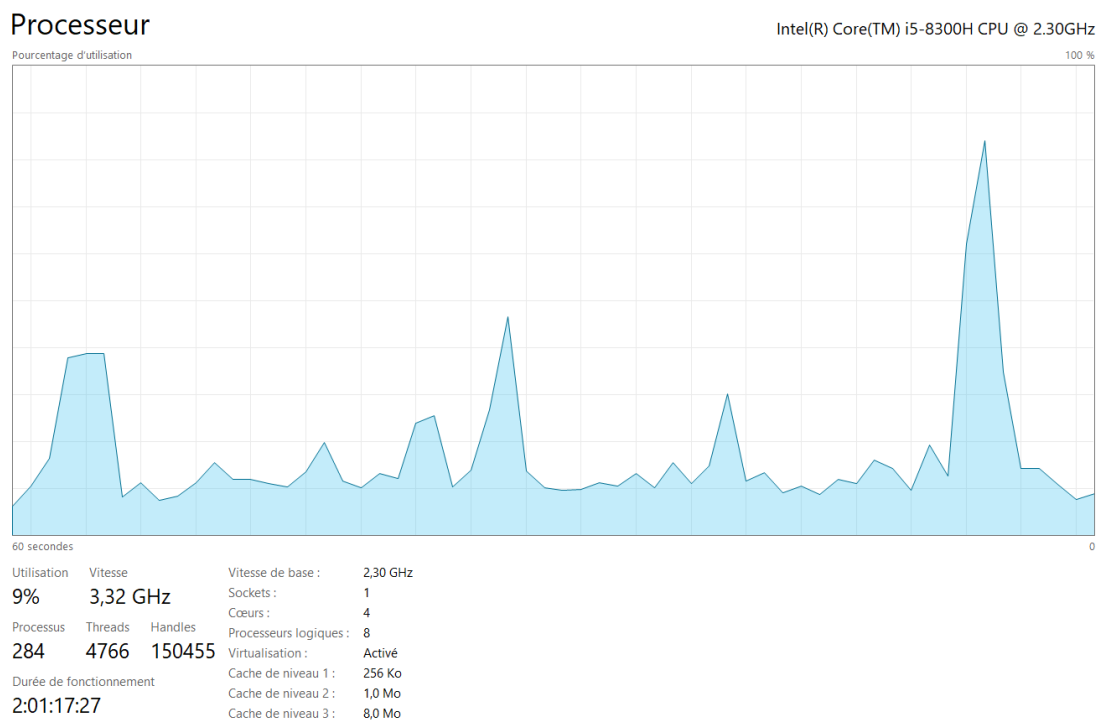
Il est toutefois important de rappeler que la gestion de ce mémoire s'observe suivant une certaine politique qui sont les suivantes :

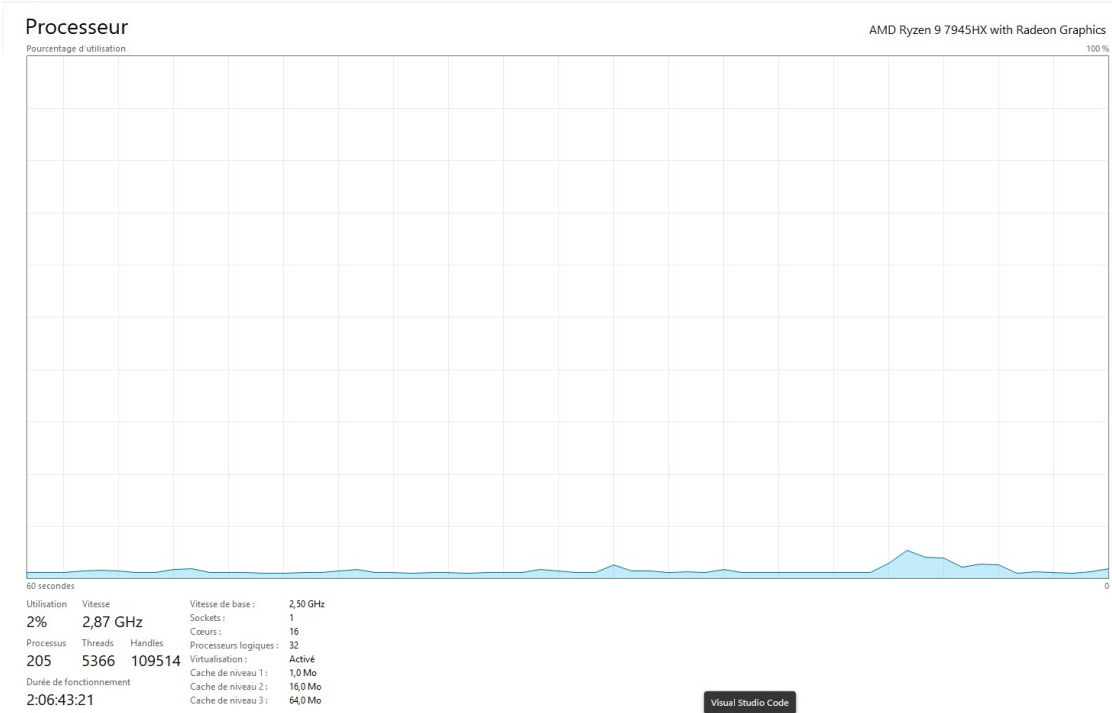
- Le cache L1 est toujours inclus dans le L2. Si une donnée est dans le L1, une copie exacte se trouve dans le L2. Le cache L2 agit ainsi comme un filet de sécurité pour le L1.
- Lorsqu'un cache est plein et qu'il doit charger de nouvelles données depuis la RAM ou le cache L3, il doit en expulser (effacer) d'autres.

Une approche différente s'observe néanmoins lorsqu'il s'agit de la gestion de la cache L3. Intel par exemple utilise une politique inclusive dans lequel une copie des caches

L1/L2 se trouve dans le L3 (ce qui peut gaspiller un peu d'espace) comparé à AMD qui utilise une approche inclusive ou les caches L1/L2 ne sont pas forcément copié (ce qui donne un gain de vitesse).

Pour les constructeurs mondiaux de processeur, optimiser la gestion de l'accès et la taille des cache L1/L2 et L3 constitue un point central dans les performances techniques des processeurs. Les ordinateurs récents les plus performants ont des espaces caches de plus en plus grande offrant des vitesses de traitement fulgurante. Mais pour y faire face, Il est nécessaire de trouver un équilibre entre la taille physique de ces mémoires, leur emplacement, leur chaleur et la disposition du CPU ce qui n'est pas une mince affaire.





3 La méthodologie expérimentale

Objectif : Analyser l'effet des changements de contexte sur les caches misses entre 2 processus/2 threads.

Cette partie de notre travail porte principalement sur l'ensemble de la démarche qui a été adopté pour mener à bien le projet.

3.1 Environnement et outils d'utilisation

Pour ce projet, nous aurons besoin comme ressources :

- Une machine sous linux (Ubuntu) intel-core i5-5400 avec 4 cœurs 2,4 GHz

Et comme outils d'analyse :

- La commande `taskset()` pour forcer l'utilisation d'un seul cœur lors de la capture
- La commande `perf()` nécessaire pour voir les statistiques de traitement des caches
- La commande `fork()` pour dupliquer le processus parent afin de créer plusieurs processus enfants
- Le procédé de création de thread selon la norme POSIX comme vus plus haut
- La commande `sched_yield()` pour provoquer les changements de contexte
- Python pour une analyse statistique exploratoire

Il est important de noter que afin de restreindre la manipulation et le monitoring du CPU au débutant, les systèmes linux ne permettent pas nativement l'utilisation de la commande `perf()`.

Pour remédier à cela, linux classe dans son fichier de configuration `/proc/Sys/kernel/perf_event` un ordre de priorité d'accès au CPU allant de -1 à 4. Plus le nombre dans ce fichier de configuration est petit, plus l'utilisateur a accès au monitoring processeur du CPU. Par défaut, ce nombre est de 4. Il suffit donc dans notre cas de modifier ce nombre en utilisant simultanément la commande `echo` et `tee` (pour contourner la permission root).

```

dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ perf stat
Error:
Access to performance monitoring and observability operations is limited.
Consider adjusting /proc/sys/kernel/perf_event_paranoid setting to open
access to performance monitoring and observability operations for processes
without CAP_PERFMON, CAP_SYS_PTRACE or CAP_SYS_ADMIN Linux capability.
More information can be found at 'Perf events and tool security' document:
https://www.kernel.org/doc/html/latest/admin-guide/perf-security.html
perf_event_paranoid setting is 4:
  -1: Allow use of (almost) all events by all users
      Ignore mlock limit after perf_event_mlock_kb without CAP_IPC_LOCK
  >= 0: Disallow raw and ftrace function tracepoint access
  >= 1: Disallow CPU event access
  >= 2: Disallow kernel profiling
To make the adjusted perf_event_paranoid setting permanent preserve it
in /etc/sysctl.conf (e.g. kernel.perf_event_paranoid = <setting>)
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ cat /proc/sys/kernel/perf_event_paranoid
4
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ echo 0 | sudo tee /proc/sys/kernel/perf_event_paranoid
[sudo] Mot de passe de dalil-linux :
0
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ cat /proc/sys/kernel/perf_event_paranoid
0
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Bureau$ perf stat

^C
Performance counter stats for 'system wide':

      245 185,72 msec cpu-clock             #    4,000 CPUs utilized
        103 987      context-switches      #   424,115 /sec
           190      cpu-migrations         #    0,775 /sec
           303      page-faults            #    1,236 /sec
    1 487 029 463      cycles                #    0,006 GHz
    884 123 094      instructions           #    0,59  insn per cycle
    173 491 865      branches              #   707,594 K/sec
      5 227 503      branch-misses         #    3,01% of all branches

    61,297948916 seconds time elapsed

```

3.2 Étapes de déroulement

3.2.1 1^{ère} étape : préparer les codes pour les processus et thread

Pour pouvoir réaliser cette analyse, nous devons capturer les résultat monitoring de notre programme en cours d'exécution. Pour que l'étude ne soit pas biaisé, nous allons réaliser la même tâche pour le cas 2 processeurs que pour le cas 2 thread : remplir et lire les éléments d'un tableau de grande taille.

Par ailleurs, pour provoquer des changements de contexte, nous allons utiliser la commande `sched_yield()`.

Point sur la commande `sched_yield()` : La commande `sched_yield` est une commande qui permet au processeur de forcer un changement de contexte. Lorsqu'elle est appelée, la tâche abandonne volontairement le CPU, le scheduler choisit une autre tâche prête ce qui provoque des commutations de contexte (qui est ce qu'on cherche). Pour pouvoir donc Contrôler ces changements de contextes et faire de l'analyse, nous allons mettre cette commande dans un boucle `for()` avec comme paramètre d'itération la constante `YIELD_FREQ`. De cette manière, pour faire varier les changements de contexte, nous allons juste réexécuter successivement le code en changeant la valeur de `YIELD_FREQ` (des valeurs allant de 1 à 1M).

```
6
7  #define SIZE 1000000
8  #define ITER 10000000
9  #define YIELD_FREQ 1000
10
11 void worker(int id)
12 {
13     int *tab = malloc(sizeof(int) * SIZE);
14
15     for(long i = 0; i < ITER; i++)
16     {
17         tab[i % SIZE] += id;
18
19         if(i % YIELD_FREQ == 0)
20             sched_yield();
21     }
22
23     free(tab);
24 }
25
26 int main()
27 {
28     pid_t pid = fork();
29
30     if(pid == 0)
31     {
32         worker(1);
33         exit(0);
34     }
35     else
36     {
37         worker(2);
38         wait(NULL);
39     }
40
41     return 0;
42 }
```

Code processus

```

1 #define YIELD_FREQ 1000
2
3 int *tab;
4
5 void *worker(void *arg)
6 {
7     long id = (long)arg;
8
9     for(long i = 0; i < ITER; i++)
10     {
11         tab[i % SIZE] += id;
12
13         if(i % YIELD_FREQ == 0)
14             sched_yield();
15     }
16
17     return NULL;
18 }
19
20 int main()
21 {
22     pthread_t t1, t2;
23
24     tab = malloc(sizeof(int) * SIZE);
25
26     pthread_create(&t1, NULL, worker, (void *)1);
27     pthread_create(&t2, NULL, worker, (void *)2);
28
29     pthread_join(t1, NULL);
30     pthread_join(t2, NULL);
31
32     free(tab);
33
34     return 0;
35 }

```

Code thread

3.2.2 2^{ème} étape : Prélèvement des données et constitution d'un datasheet

Après avoir compilé à travers les codes successivement avec le compilateur gcc, il est question d'appliquer la commande `perf stat()` (avec des paramètres) afin de capturer les données pour construire un dataset. Il s'agit de :

- Nombre de tour (avec la boucle `for()` lié à la constante `YIELD_FREQ`)
- `Cache_Load_L1` : Nombre de requête du processeur dans la cache L1
- `Cache_Miss_L1` : Nombre de caches misses Sur le cache L1
- Taux d'échec L1 : Pourcentage de perte calculé en divisant le `Cache_Load_L1` et le `Cache_Miss_L1`
- `Cache_Load_L2` : Nombre de requête du processeur dans la cache L2
- `Cache_Miss_L2` : Nombre de caches misses Sur le cache L2
- Taux d'échec L2 : Pourcentage de perte calculé en divisant le `Cache_Load_L2` et le `Cache_Miss_L2`

- Temps d'exécution
- Changement de contexte

Il s'agira de relever ces données sur plusieurs essais (10) concernant l'exécution du code des cas de 2 processus et le cas de 2 thread.

```
Taskset -c 0 perf stat -e L1-dcache-loads,L1-dcache-load-misses,
    l2_rqsts.references,l2_rqsts.miss,LLC-loads,LLC-load-misses
    ./processus
```

3.2.3 3^{ème} étape : Analyse statistique exploratoire du datasheet à travers python

Le dataset étant conçu. Nous allons utiliser le langage python à travers les bibliothèques `seaborn`, `matplotlib`, pour effectuer une analyse descriptive avec des graphiques d'illustrations.

3.2.4 4^{ème} étape : Commentaire et discussion

Ici il s'agira de discuter sur les résultats et de faire des interprétations.

3.3 Mise en œuvre

Nous avons compilé les codes de la création de 2 processus et 2 threads à travers le compilateur `gcc`.

`Gcc process.c -o process`, et `gcc thread.c -o thread`

Quelques screenshot des sorties :

```
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ gcc process_cache.c -o process
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ sudo taskset -c 0 perf stat -e context-switches,L1-dcache-load,L1-dcache-miss
es,l2_rqsts.references,l2_rqsts.miss ./process
=== Version PROCESSUS ===
Taille tableau : 10000000 entiers par processus
Iterations      : 3

[Pere PID=3685] Iteration 1 - debut
[Fils PID=3686] Iteration 1 - debut
[Pere PID=3685] Iteration 1 - fin
[Pere PID=3685] Iteration 2 - debut
[Fils PID=3686] Iteration 1 - fin
[Fils PID=3686] Iteration 2 - debut
[Pere PID=3685] Iteration 2 - fin
[Pere PID=3685] Iteration 3 - debut
[Fils PID=3686] Iteration 2 - fin
[Fils PID=3686] Iteration 3 - debut
[Pere PID=3685] Iteration 3 - fin
[Fils PID=3686] Iteration 3 - fin

[OK] Termine.

Performance counter stats for './process':

    6 075      context-switches
  1 987 310 044 L1-dcache-load
    96 581 709 L1-dcache-misses          #    4,86% of all L1-dcache accesses
   169 610 960 l2_rqsts.references
   101 143 580 l2_rqsts.miss

    2,674743260 seconds time elapsed

    2,429617000 seconds user
    0,243575000 seconds sys
```

```

dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ gcc process_cache.c -o process
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ sudo taskset -c 0 perf stat -e context-switches,L1-dcache-load,L1-dcache-miss
es,l2_rqsts.references,l2_rqsts.miss ./process
=== Version PROCESSUS ===
Taille tableau : 10000000 entiers par processus
Iterations      : 3

[Pere PID=5704] Iteration 1 - debut
[Fils PID=5705] Iteration 1 - debut
[Fils PID=5705] Iteration 1 - fin
[Fils PID=5705] Iteration 2 - debut
[Pere PID=5704] Iteration 1 - fin
[Pere PID=5704] Iteration 2 - debut
[Fils PID=5705] Iteration 2 - fin
[Fils PID=5705] Iteration 3 - debut
[Pere PID=5704] Iteration 2 - fin
[Pere PID=5704] Iteration 3 - debut
[Fils PID=5705] Iteration 3 - fin
[Pere PID=5704] Iteration 3 - fin

[OK] Termine.

Performance counter stats for './process':

   56 309 537      context-switches
  77 959 519 915    L1-dcache-load
  652 193 930      L1-dcache-misses          #    0,84% of all L1-dcache accesses
    4 046 134 896    l2_rqsts.references
   200 868 015      l2_rqsts.miss

143,146988963 seconds time elapsed

38,715480000 seconds user
104,306143000 seconds sys

```

```

dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ gcc threads_cache.c -o thread -pthread
dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ sudo taskset -c 0 perf stat -e context-switches,L1-dcache-load,L1-dcache-miss
es,l2_rqsts.references,l2_rqsts.miss ./thread
=== Version THREADS ===
Taille tableau : 10000000 entiers par thread
Iterations      : 3

[Thread 1] Iteration 1 - debut
[Thread 2] Iteration 1 - debut
[Thread 1] Iteration 1 - fin
[Thread 1] Iteration 2 - debut
[Thread 2] Iteration 1 - fin
[Thread 2] Iteration 2 - debut
[Thread 2] Iteration 2 - fin
[Thread 2] Iteration 3 - debut
[Thread 1] Iteration 2 - fin
[Thread 1] Iteration 3 - debut
[Thread 2] Iteration 3 - fin
[Thread 1] Iteration 3 - fin

[OK] Termine.

Performance counter stats for './thread':

      1 441      context-switches
   1 982 047 941    L1-dcache-load
    93 811 555      L1-dcache-misses          #    4,73% of all L1-dcache accesses
   159 217 889      l2_rqsts.references
    92 171 443      l2_rqsts.miss

2,650332322 seconds time elapsed

2,424139000 seconds user
0,223012000 seconds sys

```

```

dalil-linux@dalil-linux-HP-ProDesk-400-G3-MT:~/Documents$ sudo taskset -c 0 perf stat -e L1-dcache-loads,L1-dcache-load-misses,l2_rqsts
.references,l2_rqsts.miss,context-switches ./process
=== Version PROCESSUS ===
Taille tableau : 1000 entiers par processus
Iterations : 3

[Pere PID=4384] Iteration 1 - debut
[Fils PID=4385] Iteration 1 - debut
[Fils PID=4385] Iteration 1 - fin
[Fils PID=4385] Iteration 2 - debut
[Fils PID=4385] Iteration 2 - fin
[Fils PID=4385] Iteration 3 - debut
[Fils PID=4385] Iteration 3 - fin
[Pere PID=4384] Iteration 1 - fin
[Pere PID=4384] Iteration 2 - debut
[Pere PID=4384] Iteration 2 - fin
[Pere PID=4384] Iteration 3 - debut
[Pere PID=4384] Iteration 3 - fin

[OK] Termine.

Performance counter stats for './process':

      645 182      L1-dcache-loads
      26 857      L1-dcache-load-misses          #    4,16% of all L1-dcache accesses
     114 201      l2_rqsts.references
      68 313      l2_rqsts.miss
         1      context-switches

0,000702812 seconds time elapsed

0,001030000 seconds user
0,000411000 seconds sys

```

À chaque sortie de ces commandes, nous enregistrons dans Excel l'ensemble des données pertinentes à l'étude. Voici le résultat sous Excel :

Resultat de l'execution de 2 processus							
YIELD_SCHELD	Cache_Load_L1	Cache_Miss_L1	Taux_d'echec_L1(en %)	Cache_Load_L2	Cache_Misses_L2	Taux_d'echec_L2(en %)	Nbre_Changement_de_context
1	77956700571	772952272	0,99%	4064850835	195435598	4,807940216	57002595
10	9615412986	228443864	2,38%	684586675	162289923	23,70626378	5714360
100	2747978779	163707779	5,96%	329106139	152555455	46,35448475	576392
1000	2058342727	118034141	5,73%	255876815	159594161	62,37148176	59604
10000	1987575743	96622829	4,86%	169572173	100890584	59,49713459	6108
100000	1980784008	93937476	4,74%	156782824	89383977	57,01133244	833
1000000	1980528000	93834060	4,74%	159922908	89448221	55,9320876	969
10000000	1980727745	93972607	4,74%	156183463	89443111	57,26797785	1025
Resultat de l'execution de 2 thread							
YIELD_SCHELD	Cache_Load_L1	Cache_Miss_L1	Taux_d'echec_L1(en %)	Cache_Load_L2	Cache_Misses_L2	Taux_d'echec_L2(en %)	Nbre_Changement_de_context
1	76112745759	1042986661	1,37%	3398927227	177388195	5,218946543	57099449
10	9436369266	191207786	2,03%	612536996	149424227	24,39431871	5717288
100	2735503491	151724716	5,55%	313016447	146486492	46,79833709	577754
1000	2056988386	113673928	5,53%	242386615	147066077	60,67417419	59589
10000	1987139392	95912838	4,83%	167775010	99536531	59,32738791	6121
100000	1981245661	93781104	4,73%	157883866	91215303	57,77366954	966
1000000	1982047941	93811555	4,73%	159217889	92171443	57,89013005	1441
10000000	1981622514	93732469	4,73%	151533244	85235785	56,2489014	1351

Les données ont été importé dans python pour étude. Voici un Screenshot des Dataframes :

processus								
	YIELD_SCHELD	Cache_Load_L1	Cache_Miss_L1	Taux_d'echec_L1(en %)	Cache_Load_L2	Cache_Misses_L2	Taux_d'echec_L2(en %)	Nbre_Changement_de_contexte
0	1	77956700571	772952272	0.99	4064850835	195435598	4.81	57002595.0
1	10	9615412986	228443864	2.38	684586675	162289923	23.71	5714360.0
2	100	2747978779	163707779	5.96	329106139	152555455	46.35	576392.0
3	1000	2058342727	118034141	5.73	255876815	159594161	62.37	59604.0
4	10000	1987575743	96622829	4.86	169572173	100890584	59.50	6108.0
5	100000	1980784008	93937476	4.74	156782824	89383977	57.01	833.0
6	1000000	1980528000	93834060	4.74	159922908	89448221	55.93	969.0
7	10000000	1980727745	93972607	4.74	156183463	89443111	57.27	NaN

thread								
	YIELD_SCHELD	Cache_Load_L1	Cache_Miss_L1	Taux_d'echec_L1(en %)	Cache_Load_L2	Cache_Misses_L2	Taux_d'echec_L2(en %)	Nbre_Changement_de_contexte
0	1	76112745759	1042986661	1.37	3398927227	177388195	5.22	57099449
1	10	9436369266	191207786	2.03	612536996	149424227	24.39	5717288
2	100	2735503491	151724716	5.55	313016447	146486492	46.80	577754
3	1000	2056988386	113673928	5.53	242386615	147066077	60.67	59589
4	10000	1987139392	95912838	4.83	167775010	99536531	59.33	6121
5	100000	1981245661	93781104	4.73	157883866	91215303	57.77	966
6	1000000	1982047941	93811555	4.73	159217889	92171443	57.89	1441
7	10000000	1981622514	93732469	4.73	151533244	85235785	56.25	1351

4 Résultats et discussion

Début de l'analyse

	YIELD_SCHELD	Nbre_Changement_de_contexte
0	1	57099449
1	10	5717288
2	100	577754
3	1000	59589
4	10000	6121
5	100000	966
6	1000000	1441
7	10000000	1351

On rappelle que c'est la commande `sched_yield()` qui nous a permis ce control.
Par ailleurs, en calculant les moyennes, on constate 2 choses :

- Que les proportions des taux d'échec L1 et L2 des processus et thread sont relativement pareil.
- Que le taux d'échec L2 est beaucoup plus grand que celui des L1 bien que la moyenne des `Caches_Load` des L1 est toujours plus grande que celui des L2.

On rappelle que $\text{taux échec} = \text{Cache_Misses} / \text{Cache_Load}$

```

: mean1_proc = np.mean(processus["Taux_d'echec_L1(en %)"])
  mean2_proc = np.mean(processus["Taux_d'echec_L2(en %)"])
  mean1_thre = np.mean(thread["Taux_d'echec_L1(en %)"])
  mean2_thre = np.mean(thread["Taux_d'echec_L2(en %)"])
  print("La moyenne du taux d'echec L1 des PROCESSUS est de ",mean1_proc)
  print("La moyenne du taux d'echec L2 des PROCESSUS est de ",mean2_proc)
  print("La moyenne du taux d'echec L1 des THREAD est de ",mean1_thre)
  print("La moyenne du taux d'echec L2 des THREAD est de ",mean2_thre)

```

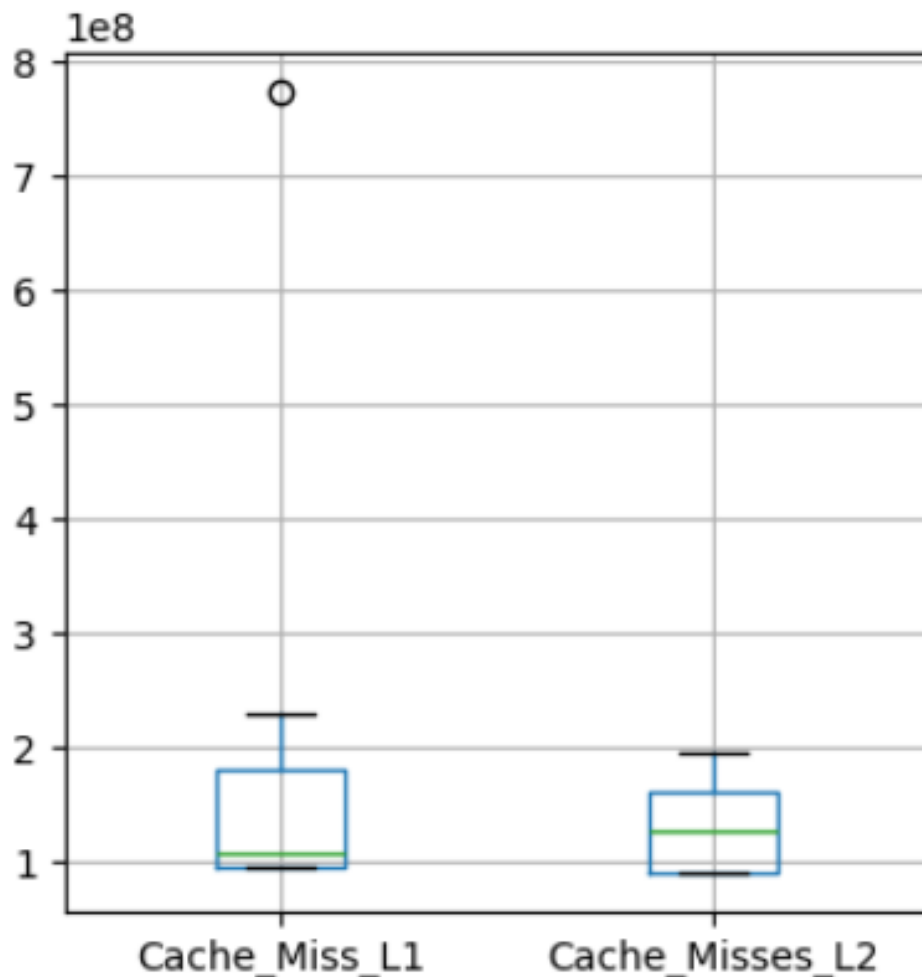
```

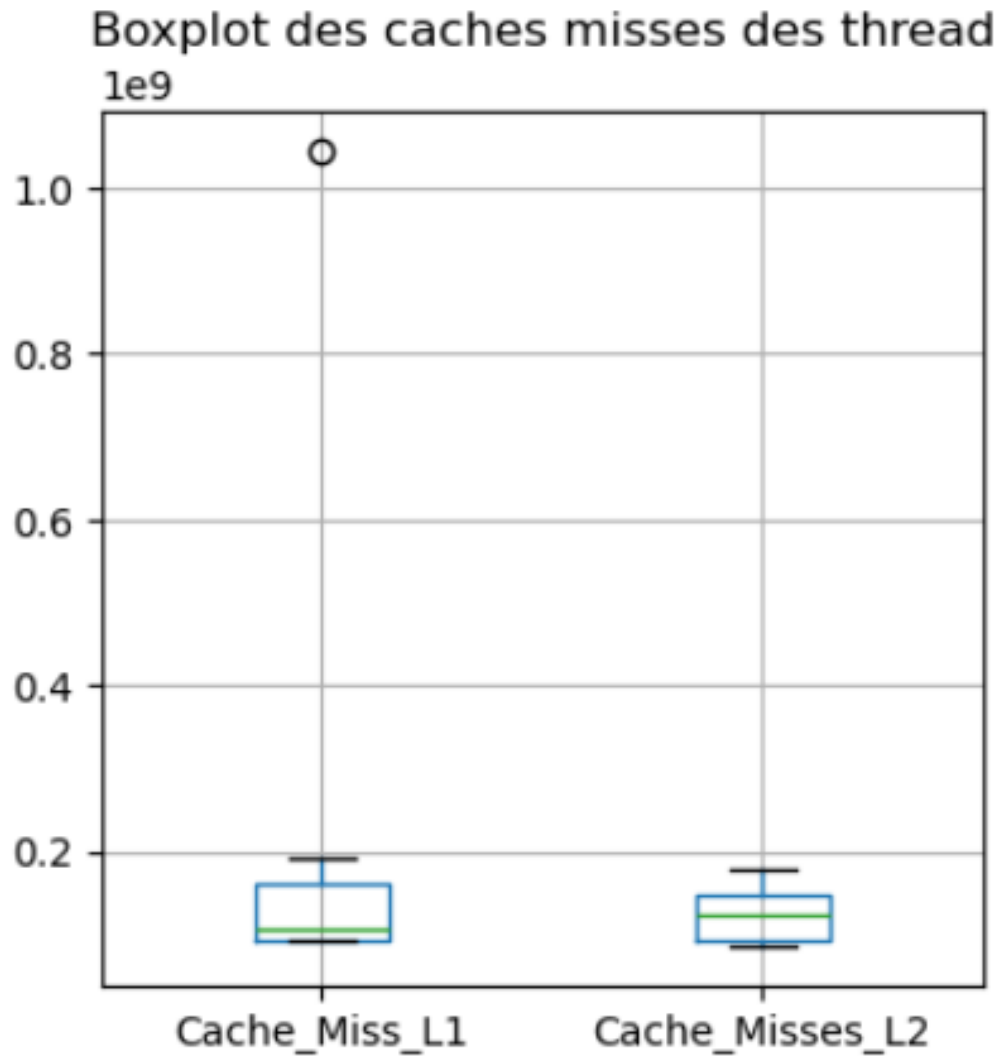
La moyenne du taux d'echec L1 des PROCESSUS est de  4.2675
La moyenne du taux d'echec L2 des PROCESSUS est de 45.86875
La moyenne du taux d'echec L1 des THREAD est de  4.1875
La moyenne du taux d'echec L2 des THREAD est de 46.04

```

Ceci s'explique par le fait que la cache L1 est beaucoup plus petite que celle L2 (254 Ko contre 4 Mo/8 Mo). Par défaut le processeur commence à chercher ces données dans la cache L1, ce qui provoque beaucoup d'accès mémoire (Cache_Load). Ce n'est que quand les données ne sont pas trouvées qu'il consulte le L2. Or dans le L2, il y a aussi les données L1. Donc le L2 a en même temps, les données du L1 et ces propres données ce qui rend le nombre de caches misses important. Il est par ailleurs

Boxplot des caches misses des processus

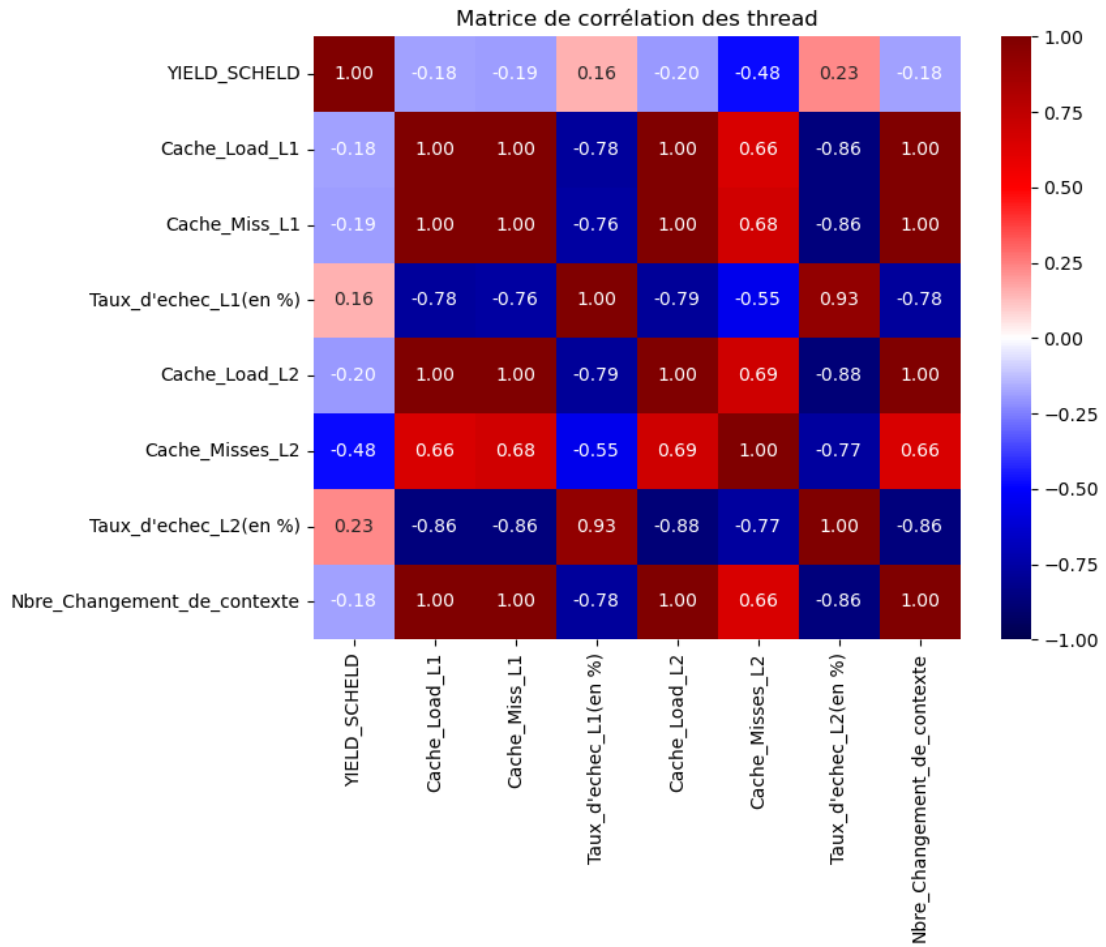




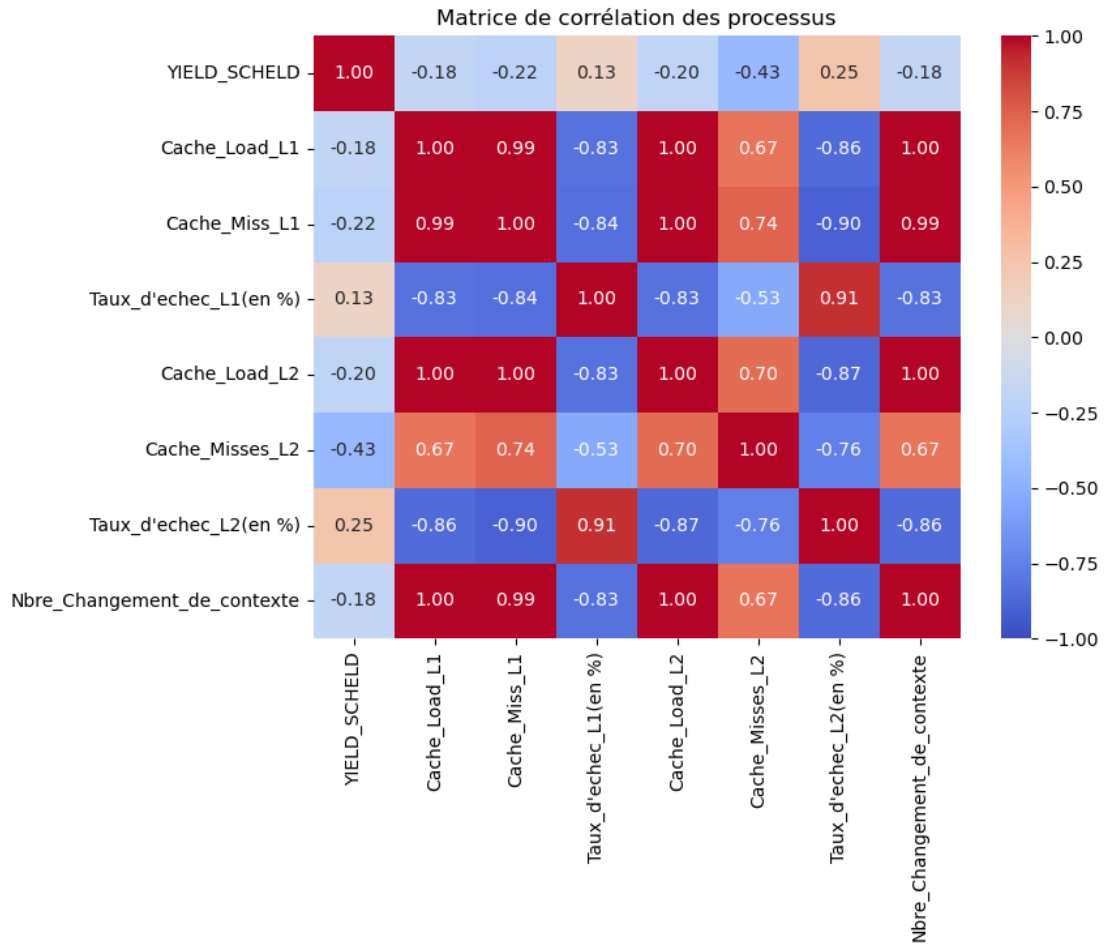
Recherche de corrélation entre les variables

On a établi le head_map (matrice de corrélation) entre les 2 datasets et voici les résultats :

- Thread



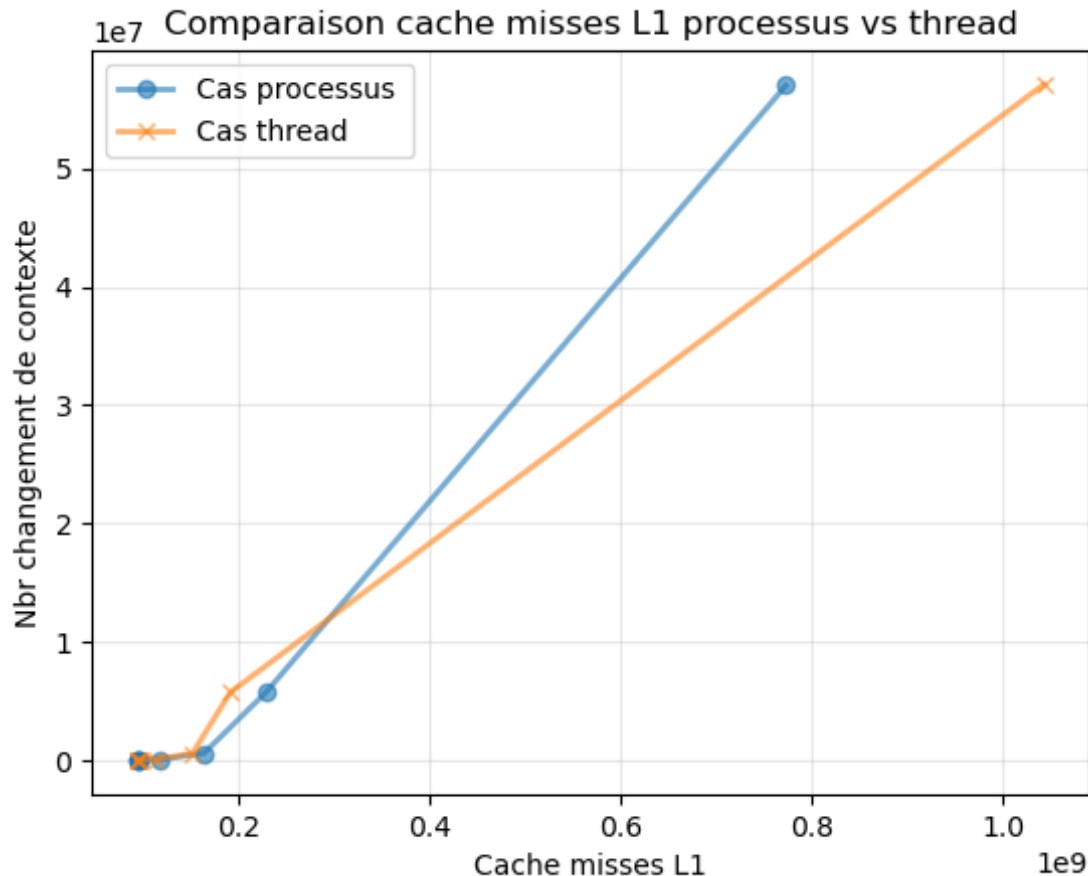
On relève des corrélations très fortes entre certaines variables caractérisées sur le graphe par la couleur rouge. Il s'agit par exemple des `Nbre_changement_contexte` vs `Cache_Load`, `Nbre_changement_contexte` vs `Cache_Misses`. Ainsi que `Cache_Load` vs `Cache_Misses`...etc.



Relation et graphe entre les variables des Thread vs processus

Pour entrer dans le point central de notre projet («Analyse du changement de contexte entre les Cache Misse L1/L2 entre 2 processus/2 thread»), nous avons mis en relation les changements de contexte vs cache_misses entre processus et thread pour mieux voir l'évolutivité.

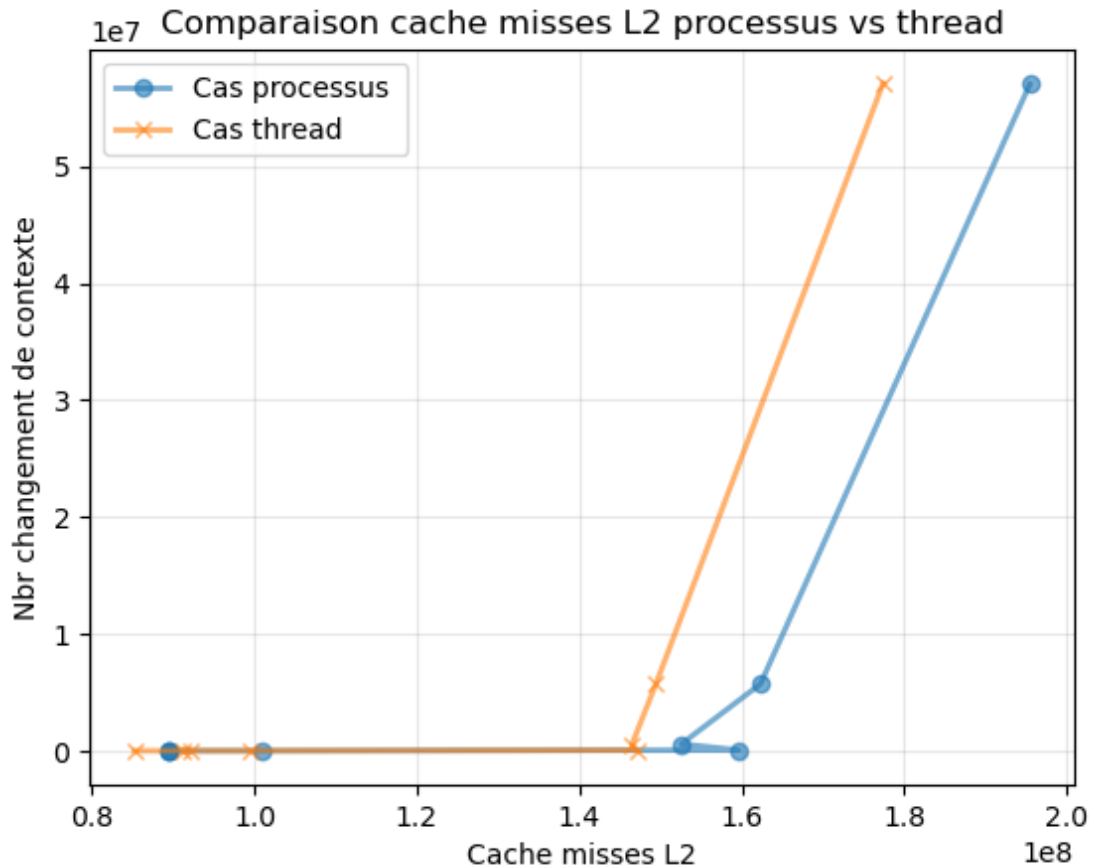
- Cas L1



On constate que malgré le fait que les courbes processus/thread sont relativement proches, il demeure que les caches misses_L1 sont plus important sur les processus que sur les threads au fur et à mesure que on augmente les changements de contexte. Ceci s'explique par le fonctionnement général d'un processus. Contrairement aux threads qui partagent le même espace mémoire au sein d'un processus, chaque processus possède ses propres structures. Cet isolement provoque une moins bonne exploitation du cache. Cela entraîne :

- **Absence de partage de données** : Deux threads travaillant sur les mêmes données utilisent le même cache (L1/L2) sans interruption. Deux processus doivent faire appel à des mécanismes lourds de communication inter-processus (IPC) et recharger les données depuis la RAM.
- **Contamination du cache (Pollution)** : Lors d'un changement de contexte entre deux processus, le processeur doit vider ses lignes de cache pour charger les données du nouveau processus. Ces données précédemment chargées sont « éjectées », ce qui provoque des cache misses obligatoires (*cold misses*) à chaque retour.
- **Fragmentation de la mémoire** : Les processus allouent la mémoire de manière complètement indépendante, augmentant la dispersion des données (*prefetching*) du processeur.

- Cas L2



L'effet du changement de contexte sur la cache_misses L2 est relativement le même que celui du L1. Ce graphique est moins illustratif que celui précède à cause de la variabilité plus important de la cache_L2 par rapport au L1 du fait de sa taille plus grande.

Ces deux niveaux de cache subissent le plus lourd tribut lors d'un changement de contexte car ils sont intégrés au plus près de chaque cœur du processeur (Core).

4.1 Discussion

La différence de performance entre les processus et les threads face aux cache misses (notamment L1/L2) permet de tirer trois conclusions majeures pour l'architecture des systèmes et le développement logiciel.

4.1.1 Le thread est l'unité reine de la performance pure

Pour les applications qui nécessitent un calcul intensif et un traitement de données massif en temps réel (comme les moteurs de jeux vidéo, le traitement vidéo ou les bases de données), le modèle multithread est indispensable.

Le gain : Vous maximisez l'efficacité des caches L1/L2 en évitant de vider et recharger la mémoire à chaque changement d'exécution.

4.1.2 Le processus est le choix de la sécurité et de la robustesse

Si les processus pénalisent le cache, ils offrent un isolement total. Si un thread plante (erreur mémoire, crash), il embarque tout le processus et les autres threads avec lui. Si un processus plante, les autres continuent de tourner.

Le compromis : Les architectures Web modernes (comme les navigateurs Chrome/-Firefox ou les serveurs Nginx) acceptent de perdre un peu en efficacité de cache (modèle multiprocesseur) pour garantir qu'un onglet ou un site corrompu ne fasse pas crasher toute l'application.

4.1.3 La localité des données dicte la vitesse moderne

Aujourd'hui, la vitesse des processeurs augmente beaucoup plus vite que celle de la mémoire RAM (problème du *Memory Wall*). Aller chercher une donnée dans la RAM est des centaines de fois plus lent que dans le cache L1.

La règle d'or : Moins un code provoque de changements de contexte de processus, plus il maintient les caches « chauds », et plus l'application est rapide.

Un peu d'histoire

La découverte que les processus génèrent plus de défauts de cache (*cache misses*) que les threads découlent des travaux pionniers en architecture des ordinateurs et en systèmes d'exploitation menés dans les années 1990 et 2000, notamment par des chercheurs comme Susan Eggers (Université de Washington) et Jean-Loup Baer. Dès 2000, d'autres chercheurs comme Calin Cascaval et David Padua ont développé des modèles pour estimer le comportement du cache par rapport aux espaces de travail parallèles.

Les chercheurs faisaient tourner des programmes identiques (comme la suite de calculs scientifiques SPLASH-2) en version « multi-processeur » et en version « multi-threads » (comme dans le cas de notre étude) pour compter exactement chaque accès mémoire. Les résultats ont montré que la « pollution du cache » (le remplacement de données utiles par celles d'un autre contexte) était radicalement plus élevée avec les processus. Ceci a eu un intérêt majeur dans le développement logiciel et l'optimisation en consommation énergétique des serveurs web ou différentes techniques d'optimisations ont été mise sur pied.

5 Conclusion

Rendu au terme de notre rapport qui portait sur : *Analyse de l'impact des Threads vs Processus sur la Consommation de Cache L1/L2*, il en ressort que nous avons évalué l'influence des changements de contexte sur les performances de la hiérarchie mémoire, en particulier sur les défauts de cache L1 et L2, dans des environnements utilisant des processus et des threads.

Pour cela, nous avons fait varier la fréquence des appels à `sched_yield()`, ce qui a permis de contrôler indirectement le nombre de changements de contexte générés par le système d'exploitation. Les mesures ont été réalisées à l'aide des compteurs matériels de performance fournis par `perf`.

Les résultats obtenus nous ont permis de constater qu'une augmentation du nombre de changements de contexte entraîne une augmentation significative des défauts de cache ceci à cause de l'exécution répétée des tâches processeur qui font perdre des données.

L'étude met également en évidence que l'impact des changements de contexte n'est pas strictement linéaire. Pour des fréquences de commutation intermédiaires, les taux d'échec observés dans les caches atteignent des valeurs particulièrement élevées. À l'inverse, lorsque les commutations deviennent extrêmement fréquentes, le coût de l'ordonnancement lui-même devient prépondérant, modifiant le comportement global du système.

La comparaison entre les exécutions basées sur des threads et celles basées sur des processus ont également montré que les processus tendent à générer davantage de perturbations dans la hiérarchie mémoire. Cette différence s'explique principalement par la séparation des espaces d'adressage des processus, alors que les threads partagent une partie importante de leurs données et de leur contexte d'exécution.

En définitive, cette étude confirme que les changements de contexte constituent un facteur important de dégradation des performances mémoire. La réduction des commutations inutiles et une gestion adaptée du parallélisme apparaissent ainsi comme des éléments essentiels pour optimiser les performances des applications modernes.